

Lecture 8 - Phân tích Code C C++

Mục lục

Lecture 8: Phân tích code C/C++ trong tài liệu	2
Vì sao cần một chương riêng phân tích code?	2
Phương pháp đọc code 6 bước	3
Cấu trúc 5 bài trong cụm này	3
Format chung của mỗi sample	4
Lưu ý về code gốc	5
Mục tiêu cuối cụm	5
8.2 Code patterns Cụm 1 (Lec 1-2)	6
Sample 1: Buffer overflow cổ điển (gets)	6
Sample 2: NULL pointer dereference	8
Sample 3: Double free	9
Sample 4: Use After Free	10
Sample 5: Integer overflow (signed)	11
Sample 6: Format string vulnerability	12
Sample 7: Race condition trên global	13
Sample 8: TOCTOU (Time of Check, Time of Use)	15
Sample 9: Integer signedness bug	16
Sample 10: Off-by-one trong string copy	17
Tóm tắt Cụm 1	18
Pattern chung rút ra	18
8.3 Code patterns Cụm 2 (Lec 3 - BMC encoding)	19
Sample 1: Array bounds với index từ symbolic	19
Sample 2: Pointer arithmetic và alias	20
Sample 3: Floating-point encoding	22
Sample 4: Float comparison subtle	23
Sample 5: SSA conversion example	24
Sample 6: Array assignment với index nondet	25
Sample 7: Memory model với multiple pointer	26
Sample 8: Encoding integer division	27
Tóm tắt Cụm 2	28
Pattern lập harness BMC tốt	28
8.4 Code patterns Cụm 3 (Lec 4 - Concurrency)	29
Sample 1: Race trên shared variable	29
Sample 2: Atomicity violation	30
Sample 3: Deadlock cổ điển	31
Sample 4: Lost wakeup	33

Sample 5: Memory model (TSO surprise)	34
Sample 6: ABA problem trong lock-free	36
Sample 7: Condition race trong initialization	37
Sample 8: False sharing performance bug	39
Tóm tắt Cụm 3	40
Pattern an toàn cho concurrency	40
8.5 Code patterns Cụm 4 (Lec 5 - Testing và Fuzzing)	41
Sample 1: Coverage demo program	41
Sample 2: Code có MC/DC challenge	42
Sample 3: Fuzz target naïve	43
Sample 4: Symbolic execution example	44
Sample 5: BMC for test generation	45
Sample 6: Fuzz harness cho parser	47
Tóm tắt Cụm 4	49
Pattern fuzz harness tốt	49
Combine BMC + Fuzz	50
8.6 Phân tích Exercise Set 2 với tool	51
Bài 1: sprintf buffer overflow	51
Bài 2: Integer overflow trong calloc	52
Bài 3: Uninitialized variable	54
Bài 4: Off-by-one trong loop	55
Bài 5: Race condition đơn giản	56
Bài 6: Use-after-free trong linked list	57
Bài 7: Struct padding và alignment	58
Bảng tóm tắt 7 bài	59
CI workflow tích hợp	60
Tóm tắt cụm 8 (Code Analysis)	61

Lecture 8: Phân tích code C/C++ trong tài liệu

Tóm tắt một dòng: Cụm này quay lại từng đoạn code C/C++ xuất hiện trong các cụm trước (Lec 1-5) và các đề bài tập, đọc kỹ từng dòng, chỉ ra cụ thể chỗ sai, giải thích cơ chế lỗi, đề xuất cách viết lại, và minh họa cách CBMC/ESBMC bắt được bug nếu có. Mục tiêu: nâng cấp khả năng **đọc code C/C++ kiểu kỹ sư an ninh**, không chỉ kiểu lập trình viên thông thường.

Vì sao cần một chương riêng phân tích code?

Trong 7 cụm trước, code xuất hiện rải rác làm minh họa cho concept. Người đọc thường tập trung vào concept, lướt qua code. Nhưng:

1. **Bug thật nằm trong code thật:** hiểu concept “buffer overflow” khác xa với việc **đọc đúng** code có buffer overflow và chỉ ra dòng nào sai.
2. **Code C/C++ trong tài liệu mặc định cô đọng:** nhiều đoạn lược bỏ header, error handling, để focus vào điểm minh họa. Người đọc cần học cách **lấp những lỗ hổng này** trong đầu khi review code production.
3. **Mỗi đoạn code có nhiều bug, không chỉ một:** tài liệu thường chỉ một bug chính. Nhưng cùng đoạn code có thể còn 2-3 bug khác mà người đọc cần thấy.

4. **Code review là kỹ năng practical:** lý thuyết BMC không thay được khả năng human reviewer đọc 200 dòng C và nhận ra UAF tiềm ẩn.

Chương này tổng hợp **toàn bộ pattern code** từ Lec 1-5, đọc chậm, comment chi tiết, đề xuất tool verify.

Phương pháp đọc code 6 bước

Mỗi đoạn code, tôi sẽ áp dụng quy trình sau:

Bước 1: Đọc lần một, hiểu chức năng

Code đang **cố làm gì**? Trước khi tìm bug, phải hiểu intent. Bug = “code làm khác intent”.

Bước 2: Liệt kê assumption ngầm

Code C/C++ thường có **assumption ngầm** mà không kiểm tra: - Input size phù hợp với buffer. - Pointer không null. - Integer không overflow. - File tồn tại. - Quyền đọc/ghi đã có.

Mỗi assumption không kiểm tra = một bug tiềm năng.

Bước 3: Tìm bug theo checklist

Đi qua các lớp lỗ hổng quen thuộc: - Memory: buffer overflow, UAF, double free, memory leak. - Integer: overflow, sign error, narrowing conversion. - Logic: race condition, TOCTOU, off-by-one. - Crypto: hardcoded key, weak random, predictable token. - Injection: SQL, command, format string. - Auth: missing check, IDOR.

Bước 4: Phân loại nghiêm trọng

Bug tìm được thuộc loại nào theo CVSS? - Critical: RCE, full data leak. - High: privilege escalation, partial data leak. - Medium: DoS, info disclosure. - Low: configuration weakness.

Quyết định “fix ngay” hay “backlog”.

Bước 5: Viết fix

Fix phải: - Đúng (không introduce bug mới). - Tối thiểu (không thay đổi semantics khác). - Có comment giải thích vì sao. - Có test (regression).

Bước 6: Verify với tool

CBMC/ESBMC trên code mới: chứng minh bug đã fix. SAST trên codebase: pattern không lặp lại nơi khác.

Cấu trúc 5 bài trong cụm này

Bài	Nguồn code phân tích	Số sample	Lớp bug
8.2 Code patterns Cụm 1	Lec 1-2 (intro, vulnerabilities)	~10	Buffer overflow, UAF, integer overflow, format string, race

Bài	Nguồn code phân tích	Số sample	Lớp bug
8.3 Code patterns Cùm 2	Lec 3 (BMC, SMT encoding)	~8	Bug để demo BMC bắt: array bounds, pointer, float
8.4 Code patterns Cùm 3	Lec 4 (concurrency)	~8	Data race, atomicity, deadlock, memory model
8.5 Code patterns Cùm 4	Lec 5 (testing, fuzzing)	~6	Coverage demo, fuzzing target
8.6 Exercise analysis	Exercise Set 2 (7 bài)	7	Mix: memory, integer, struct alignment

Format chung của mỗi sample

Mỗi sample được trình bày theo template:

Sample N: <tên ng n>

Ngu n: Lec X, mục Y.

Mục đích trong tài liệu g c: minh hoạ <concept>.

Code (cleaned up):

```
```c
... code đã làm sạch ...
```
```

Đọc nhanh (1-2 câu mô t intent).

Bug tìm được:

1. **Bug chính**: ...
2. **Bug phụ**: ...

Cơ ch:

gi i thích step-by-step cách bug x y ra, kèm memory layout/state n u c n.

Fix gợi ý:

```
```c
... code đã s a ...
```
```

Verify với CBMC (n u áp dụng):

```
```bash
cbmc sample.c --bounds-check ...
```
```

Output mong đợi.

Bài học: 1-2 câu ch t.

Lưu ý về code gốc

Code trong tài liệu nguồn thường:

- **Thiếu header:** `#include <stdio.h>`, `#include <string.h>` không có. Tôi sẽ thêm cho compile được.
- **Macro/định nghĩa ẩn:** ví dụ `nondet_int()` không khai báo, hiểu ngầm là input bất kỳ.
- **Compilation warning:** `mixed declaration`, `void main()`, `implicit int`. Sẽ giữ lại để chỉ ra warning là **bug pattern**.
- **OCR artifact:** kí tự `—`, `□` từ PDF extraction. Sẽ clean.

Tất cả code đã được tôi reformat để compile được với `gcc -std=c99` hoặc `clang -fsyntax-only`. Code “as-is” trong tài liệu có thể không compile, đó là điểm thứ nhất cần chú ý khi review code published.

Mục tiêu cuối cụm

Sau khi đọc hết 5 bài, bạn sẽ:

- Nhớ ~40 code pattern xuất hiện trong tài liệu, biết bug của mỗi pattern.
- Đọc một đoạn C/C++ 50 dòng và liệt kê được 3-5 issue trong 10 phút.
- Viết được fix cho mỗi loại bug, kèm explain.
- Biết tool nào (CBMC, ASan, fuzzer) phù hợp cho từng loại.

Đây là kỹ năng **code review formal** cho security.

Sẵn sàng? Bắt đầu với **bài 8.2: Code patterns Cụm 1**.

8.2 Code patterns Cụm 1 (Lec 1-2)

Tóm tắt một dòng: Cụm 1 giới thiệu vocabulary Software Security và liệt kê các lớp lỗ hổng implementation phổ biến. Mỗi lớp có 1-2 code sample minh họa. Bài này lấy từng sample đọc lại, comment chi tiết, giải thích state khi crash, và viết lại an toàn.

Sample 1: Buffer overflow cổ điển (gets)

Nguồn: Lec 1, mục Buffer Overflow / Stack Smashing. **Mục đích trong tài liệu gốc:** minh họa lý do `gets()` bị deprecated từ C11.

Code (cleaned up):

```
#include <stdio.h>
#include <stdlib.h>

int x = 0;

int getPassword() {
    char buf[4];
    printf("Enter password: ");
    gets(buf); // (1) BUG
    return strcmp(buf, "ML");
}

int main() {
    x = getPassword();
    if (x) {
        printf("Access Denied\n");
        exit(0);
    }
    printf("Access Granted\n");
    return 0;
}
```

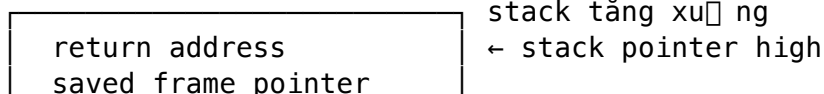
Đọc nhanh: chương trình so sánh password user nhập với chuỗi "ML". Nếu khớp, granted.

Bug tìm được:

1. **Bug chính (Critical):** `gets(buf)` không có bound check. Nhập > 3 ký tự = overflow buffer char `buf[4]`, ghi đè stack frame.
2. **Bug phụ 1 (Medium):** thiếu `#include <string.h>` cho `strcmp`. Compile warning nhưng vẫn link.
3. **Bug phụ 2 (Medium):** password compare bằng `strcmp` non-constant-time, timing attack.
4. **Bug phụ 3 (Low):** password hardcoded "ML" trong source code, lộ trong binary.
5. **Bug phụ 4 (Low):** `int x = 0` biến toàn cục không cần thiết, dễ bị race nếu multi-thread.

Cơ chế bug chính:

Stack layout trước `gets()`:



```
| buf[3]  buf[2]  buf[1]  buf[0] | ← buf points here
```

Nhập 100 ký tự, gets ghi từ &buf[0] lên cao. Sau 4 byte tràn buf. Sau 8-12 byte tràn saved FP. Sau 12-20 byte ghi đè **return address**. Khi getPassword return, CPU jump tới địa chỉ attacker chọn.

Đây là pattern **stack smashing** kinh điển. Modern OS có ASLR + stack canary giảm tỉ lệ thành công, nhưng gets vẫn là bug undefined behavior.

Fix gợi ý:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define PW_BUF_SIZE 64
#define PASSWORD_HASH "... " // bcrypt hash, không phải plaintext

int getPassword(void) {
    char buf[PW_BUF_SIZE];
    printf("Enter password: ");
    if (fgets(buf, sizeof(buf), stdin) == NULL) return -1;
    buf[strcspn(buf, "\n")] = 0; // strip newline
    // Constant-time compare with bcrypt verify (không show ở đây)
    return verify_bcrypt(buf, PASSWORD_HASH);
}

int main(void) {
    int ok = getPassword();
    if (!ok) {
        puts("Access Denied");
        return 1;
    }
    puts("Access Granted");
    return 0;
}
```

Thay đổi: - gets → fgets với size limit. - Buffer 4 → 64 byte phù hợp password thật. - Compare bằng bcrypt verify (constant-time, không leak timing). - Không hardcode plaintext. - void main → int main(void) chuẩn ISO C.

Verify với CBMC:

```
cbmc original.c --bounds-check
```

Output (nếu compile được không có gets):

```
[getPassword.array_bounds.1] line 6 array `buf' upper bound: FAILURE
```

CBMC bắt được vì gets được model là “writes unbounded length”. Lý tưởng nhất, replace gets bằng harness `__CPROVER_input` để test mọi input length.

Bài học: gets() đã bị remove khỏi C11. Mọi compiler hiện đại warn nếu thấy. Code chứa gets = code legacy, cần migrate ngay.

Sample 2: NULL pointer dereference

Nguồn: Lec 1, mục NULL Pointer Dereference.

Code (cleaned up):

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    double *p = NULL;
    int n = 8;
    for (int i = 0; i < n; ++i) {
        p[i] = (double)i;    // (1) BUG
    }
    return 0;
}
```

Đọc nhanh: cấp pointer NULL rồi ghi vào index 0..7.

Bug tìm được:

1. **Bug chính (Critical):** p được gán NULL, p[i] dereference NULL pointer $\square$ SIGSEGV.
2. Có thể tác giả định ý p = malloc(n * sizeof(double)) nhưng quên.

Cơ chế:

p = NULL = 0x00000000

p[i] = 0x00000000 + i*sizeof(double) = 0x00000000..0x00000038

Truy cập page 0 trên modern OS $\square$ page fault $\square$ SIGSEGV $\square$ crash.

Chú ý: trong embedded không có MMU, write vào address 0 không crash mà **silently corrupt** memory map (vector table). Bug nguy hiểm hơn.

Fix:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 8;
    double *p = malloc(n * sizeof(double));
    if (p == NULL) {
        fprintf(stderr, "malloc failed\n");
        return 1;
    }
    for (int i = 0; i < n; ++i) {
        p[i] = (double)i;
    }
    // ... use p ...
    free(p);
    return 0;
}
```

Quan trọng: **luôn check return của malloc**. Nhiều legacy code ignore, gây bug khi memory pressure.

Verify với CBMC:

```
cbmc null.c --pointer-check
```

Output:

[main.pointer_dereference.1] line 7 dereference failure: pointer NULL: FAILURE

CBMC bắt ngay.

Bài học: pointer must check NULL trước khi dereference. Sanitizer ASan cũng bắt bug này khi chạy test.

Sample 3: Double free

Nguồn: Lec 1, mục Double Free.

Code (cleaned up):

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    char *ptr = malloc(sizeof(char));
    if (ptr == NULL) return -1;
    *ptr = 'a';
    free(ptr);
    // ... some code ...
    free(ptr); // (1) BUG: double free
    return 0;
}
```

Đọc nhanh: cấp, dùng, free, rồi free lần thứ hai.

Bug:

1. **Bug chính (High):** free(ptr) lần hai. UB theo chuẩn C.

Cơ chế:

Modern glibc/jemalloc detect double free và abort process. Older or custom allocator có thể **corrupt heap metadata**, attacker exploit để arbitrary write.

Lịch sử: nhiều CVE PHP, Firefox từ pattern này. Glibc 2.10+ có “tcache” để detect, nhưng không 100%.

Fix:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    char *ptr = malloc(sizeof(char));
    if (ptr == NULL) return -1;
    *ptr = 'a';
    free(ptr);
    ptr = NULL; // QUAN TRỌNG: gán NULL sau free
    // ... some code ...
    free(ptr); // OK: free(NULL) là no-op theo chuẩn
```

```
    return 0;
}
```

Pattern “free + set NULL” giảm rủi ro. Hoặc dùng macro:

```
#define SAFE_FREE(p) do { free(p); (p) = NULL; } while (0)

SAFE_FREE(ptr);
SAFE_FREE(ptr); // OK, free(NULL) no-op
```

Verify với CBMC:

```
cbmc double_free.c --memory-leak-check --pointer-check
```

[main.invalid_pointer.1] line 10 free argument has offset zero: FAILURE

CBMC track allocation history, bắt được double free.

Bài học: ngay sau free(p), gán p = NULL. Smart pointer (std::unique_ptr trong C++) loại bỏ class bug này hoàn toàn.

Sample 4: Use After Free

Nguồn: Lec 1, mục UAF.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(sizeof(int));
    if (p == NULL) return -1;
    *p = 42;
    free(p);
    *p = 100; // (1) BUG: UAF
    printf("%d\n", *p); // (2) BUG: UAF read
    return 0;
}
```

Bug:

1. **Bug chính (Critical):** *p = 100 sau khi free(p). Pointer dangling.
2. **Bug phụ:** printf("%d", *p) cũng UAF.

Cơ chế:

Sau free(p), glibc đưa block vào free list (tcache hoặc fastbin). Nếu allocation tiếp theo lấy lại block đó:

```
int *q = malloc(sizeof(int)); // có thể trỏ vào cùng địa chỉ p cũ
*q = 999;
```

Lúc này *p và *q cùng địa chỉ. Sửa *p = sửa *q. Attacker exploit: control content của block sau free → control behavior chương trình.

CVE thực tế: Use-After-Free trong browser engine (Chrome, Firefox) là loại CVE phổ biến nhất, ~50% memory bug class.

Fix:

```
free(p);
p = NULL;
// ... mọi access tới p phải i null check trước ...
if (p) printf("%d\n", *p);
```

Hoặc thiết kế lại với RAII / smart pointer trong C++.

Verify với ASan (chính xác hơn CBMC cho UAF):

```
clang -fsanitize=address uaf.c -o uaf
./uaf
```

Output:

```
==1234==ERROR: AddressSanitizer: heap-use-after-free on address 0x...
```

Bài học: UAF là 1 trong 4 lớp memory bug nguy hiểm nhất (cùng OOB, NULL, double free). Modern C++ với smart pointer giảm rủi ro 90%.

Sample 5: Integer overflow (signed)

Nguồn: Lec 1, mục Integer Overflow.

Code:

```
#include <stdio.h>
#include <stdlib.h>

void allocate_buffer(int size) {
    if (size < 0) return;
    char *buf = malloc(size + 1); // (1) BUG
    if (!buf) return;
    // ... use buf ...
    free(buf);
}
```

Bug:

1. **Bug chính (High):** size + 1 overflow nếu size == INT_MAX. INT_MAX + 1 = INT_MIN (UB trong signed). malloc(INT_MIN) → size_t cast → giá trị khổng lồ → malloc fail hoặc cấp 4 GB.
2. **Bug phụ:** không return code chỉ ra fail.

Cơ chế:

INT_MAX = $2^{31} - 1 = 2147483647$. INT_MAX + 1 overflow: - Trên hầu hết compiler: wrap to INT_MIN = $-2^{31} = -2147483648$. - Theo C standard: **undefined behavior**, compiler có thể assume “không xảy ra” và optimize ra code không expected.

malloc((size_t)(-2147483648)) = malloc(2147483648UL on 32-bit hoặc 18446744071562067968UL on 64-bit). Số quá lớn → malloc return NULL.

Sau đó *buf (chưa check NULL) → SIGSEGV.

Fix:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int allocate_buffer(int size) {
    if (size < 0 || size >= INT_MAX) return -1; // explicit upper bound

    // Hoặc dùng builtin overflow check (GCC, Clang):
    size_t total;
    if (__builtin_add_overflow((size_t)size, 1, &total)) return -1;

    char *buf = malloc(total);
    if (!buf) return -1;
    // ... use buf ...
    free(buf);
    return 0;
}
```

__builtin_add_overflow (GCC/Clang): trả về true nếu overflow, false nếu OK. Type-aware.

Verify với CBMC:

```
cbmc int_overflow.c --signed-overflow-check --unsigned-overflow-check
```

Output:

[allocate_buffer.overflow.1] line 5 arithmetic overflow on signed + in size + 1: FAILURE

Bài học: bất kỳ arithmetic trên integer untrusted phải có overflow check. UBSan runtime + CBMC static = defense in depth.

Sample 6: Format string vulnerability

Nguồn: Lec 1, mục Format String.

Code:

```
#include <stdio.h>

void log_message(char *user_input) {
    printf(user_input); // (1) BUG
}

int main(int argc, char *argv[]) {
    if (argc > 1) log_message(argv[1]);
    return 0;
}
```

Bug:

1. **Bug chính (Critical):** printf(user_input) cho phép user control format string. Input %s%s%s%s đọc stack □ info leak. Input %n ghi vào address trên stack □ arbitrary write.

Cơ chế:

`printf("%s")` mong đợi 1 argument trên stack tại offset cố định. Nếu user input `%s`, `printf` đọc value tại offset đó (giá trị từ stack frame), treat as pointer, dereference. Leak data hoặc crash.

`%n` ghi số character đã in ra integer pointer trên stack. Pattern `AAAA%n` ghi 4 vào địa chỉ `0x41414141`.

Lịch sử: WU-FTPD 2.6.0 (2000) bị exploit qua format string trong site command. 100,000+ server compromise.

Fix:

```
#include <stdio.h>

void log_message(char *user_input) {
    printf("%s", user_input);    // ĐÚNG: format string là literal
}
```

Một câu sửa. Quy tắc: **format string LUÔN là string literal**, không bao giờ là biến.

Compiler GCC/Clang có warning:

warning: format not a string literal and no format arguments [-Wformat-security]

Enable `-Wformat-security -Werror=format-security` trong build.

Bài học: format string bug rất dễ tránh (1 line fix) nhưng vẫn xuất hiện trong legacy code. Modern static analyzer bắt 100%.

Sample 7: Race condition trên global

Nguồn: Lec 1, mục Race Condition (intro).

Code:

```
#include <pthread.h>
#include <stdio.h>

int counter = 0;

void* increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++;    // (1) BUG
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);    // expect 2000000
    return 0;
}
```

Bug:

1. **Bug chính (High):** counter++ không atomic. Hai thread race $\square$ kết quả ngẫu nhiên $< 2\,000\,000$.

Cơ chế:

counter++ thực ra là 3 instruction:

```
LOAD  counter -> reg
INC   reg
STORE reg -> counter
```

Thread A: LOAD (giá trị 100) $\square$ INC (101) $\square$... bị preempt ... Thread B: LOAD (vẫn 100) $\square$ INC (101) $\square$ STORE (101). Thread A: tiếp tục STORE (101).

Hai increment chỉ tăng 1. Lost update.

Fix 1 (mutex):

```
#include <pthread.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void* increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Fix 2 (atomic):

```
#include <stdatomic.h>

atomic_int counter = 0;

void* increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        atomic_fetch_add(&counter, 1);
    }
    return NULL;
}
```

Atomic nhanh hơn mutex cho operation đơn giản (single int).

Verify với CBMC:

```
cbmc race.c --pthread --unwind 5
```

Output:

```
[main.assertion.1] line 16 assertion counter == 2000000: FAILURE
```

CBMC khám phá interleaving, tìm được schedule khiến counter < target.

Verify với ThreadSanitizer:

```
clang -fsanitize=thread race.c -o race
./race
```

Output:

WARNING: ThreadSanitizer: data race on counter

Bài học: mọi shared variable phải hoặc atomic, hoặc protected by lock. Single-thread $\square$ multi-thread refactor là moment cần audit race kỹ.

Sample 8: TOCTOU (Time of Check, Time of Use)

Nguồn: Lec 1, mục TOCTOU.

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

void open_file(const char *path) {
    struct stat st;
    if (stat(path, &st) == 0 && S_ISREG(st.st_mode)) {
        // (1) GAP between check and use
        int fd = open(path, O_RDONLY); // (2) BUG
        // ... process file ...
        close(fd);
    }
}
```

Bug:

1. **Bug chính (High):** giữa stat và open, attacker swap file (symlink attack). stat thấy regular file, nhưng open thực sự open file khác.

Cơ chế:

```
T1: open_file("data.txt")
T1: stat("data.txt", &st) → regular file (passes check)
T2: rm data.txt
T2: ln -s /etc/passwd data.txt // attacker swap to symlink
T1: open("data.txt", O_RDONLY) → opens /etc/passwd!
```

Nếu service chạy as root và sau đó print file content, attacker đọc được /etc/passwd.

Fix: dùng O_NOFOLLOW hoặc fstat trên fd đã open:

```
int fd = open(path, O_RDONLY | O_NOFOLLOW);
if (fd < 0) return;
struct stat st;
if (fstat(fd, &st) == 0 && S_ISREG(st.st_mode)) {
```

```

    // ... process file ...
}
close(fd);

```

O_NOFOLLOW: open fail nếu path là symlink. fstat(fd): stat trên file descriptor đã mở, không thể swap.

CVE thực tế: nhiều CVE Unix kernel utility (logrotate, cron, package manager) từ pattern này.

Bài học: TOCTOU race khó test (hiếm trigger ngẫu nhiên), nhưng attacker triggered được 99% time qua filesystem operations. Pattern fix: dùng file descriptor (mở 1 lần), không reopen.

Sample 9: Integer signedness bug

Nguồn: Lec 1, mục Integer signedness.

Code:

```

#include <string.h>
#include <stdlib.h>

int copy_data(char *dst, const char *src, int len) {
    if (len > 100) return -1; // (1) check upper bound
    memcpy(dst, src, len);    // (2) BUG khi len < 0
    return 0;
}

```

Bug:

1. **Bug chính (High):** len signed int. Negative len pass check len > 100 (vì -1 < 100). Nhưng memcpy(dst, src, len) có len là size_t (unsigned). Cast (size_t)(-1) = SIZE_MAX (~18 quintillion).
2. memcpy với khổng lồ size → buffer overflow + crash.

Cơ chế:

```

int len = -1;
if (len > 100) ... // false: -1 không > 100
memcpy(dst, src, len); // len cast to size_t = 0xFFFFFFFFFFFFFFFF
// memcpy copy 2^64 byte → segfault sau khi đề memory rỗng

```

Đây là pattern “**signed comparison + unsigned use**”, rất phổ biến trong code C cũ.

Fix:

```

int copy_data(char *dst, const char *src, size_t len) { // size_t, không int
    if (len > 100) return -1;
    memcpy(dst, src, len);
    return 0;
}

```

Hoặc nếu phải dùng int (API constraint):

```

int copy_data(char *dst, const char *src, int len) {
    if (len < 0 || len > 100) return -1; // bounds check cả hai phía
    memcpy(dst, src, (size_t)len);
}

```



```
    return 0;
}
```

Verify với CBMC:

```
cbmc signedness.c --signed-overflow-check --bounds-check
```

CBMC bắt nếu có harness test với len âm.

Bài học: dùng `size_t` cho size, `ssize_t` cho size có thể negative (e.g. return value của `read`). Tránh `int` cho buffer size. Compile với `-Wsign-compare` để compiler cảnh báo signed/unsigned mismatch.

Sample 10: Off-by-one trong string copy

Nguồn: Lec 1, mục Buffer Overflow variant.

Code:

```
#include <string.h>

void copy_name(char *dst, const char *src) {
    int i;
    for (i = 0; src[i] != '\0'; i++) {
        dst[i] = src[i];
    }
    dst[i] = '\0'; // (1) potential BUG
}
```

Bug:

1. **Bug (Medium-High):** nếu `src` length = buffer size của `dst`, không có terminator slot. Off-by-one overflow.

Cơ chế:

```
char src[] = "1234"; // length 5 (4 char + '\0')
char dst[4]; // 4 byte
copy_name(dst, src);
// Loop: dst[0]='1', dst[1]='2', dst[2]='3', dst[3]='4'
// dst[4] = '\0' ← out of bounds!
```

`dst[4]` ghi 1 byte sau buffer. Có thể đè saved register, return address. Subtle bug khó detect.

Fix với explicit buffer size:

```
int copy_name(char *dst, size_t dst_size, const char *src) {
    if (dst_size == 0) return -1;
    size_t i;
    for (i = 0; i < dst_size - 1 && src[i] != '\0'; i++) {
        dst[i] = src[i];
    }
    dst[i] = '\0';
    return (src[i] == '\0') ? 0 : -1; // -1 nếu u truncated
}
```

Pattern: **hàm string nhận size của destination.**

Hoặc dùng `snprintf` (chuẩn ISO C):

```
snprintf(dst, dst_size, "%s", src);
```

Tự động truncate và terminate.

Verify với CBMC:

```
cbmc offbyone.c --bounds-check
```

Bài học: tránh tự viết string function. Dùng `snprintf`, `strncpy` (BSD), hoặc trong C++ thì `std::string`.

Tóm tắt Cụm 1

| Sample | Lớp bug | Tool detect tốt nhất |
|--------------------|-----------------|--|
| 1 gets() | Buffer overflow | Compiler warning + CBMC + ASan |
| 2 NULL deref | Memory | CBMC + ASan |
| 3 Double free | Memory | ASan + CBMC |
| 4 UAF | Memory | ASan |
| 5 Integer overflow | Integer | UBSan + CBMC <code>-signed-overflow-check</code> |
| 6 Format string | Injection | Compiler <code>-Wformat-security</code> |
| 7 Race condition | Concurrency | TSan + CBMC <code>-pthread</code> |
| 8 TOCTOU | Concurrency | Manual review + ESBMC |
| 9 Signedness | Integer | Compiler <code>-Wsign-compare</code> + UBSan |
| 10 Off-by-one | Buffer | CBMC + ASan |

10 sample này = ~80% memory/integer bug class.

Pattern chung rút ra

Sau khi đọc 10 sample, có thể rút **pattern lặp lại**:

1. **Tin tưởng input**: code mặc định size phù hợp, pointer non-null, integer trong range.
2. **Mix signed/unsigned**: int dùng cho size, size_t dùng cho memcpy.
3. **Thiếu cleanup**: free nhưng không set NULL, mở fd không close.
4. **API legacy**: gets, sprintf, strcpy thay vì safe variant.
5. **Format string injection**: trực tiếp pass user input.

Code review checklist 5 điểm này giúp catch ~70% bug trong code C kế thừa.

Tiếp theo: 8.3 Code patterns Cụm 2 (BMC encoding)

8.3 Code patterns Cùm 2 (Lec 3 - BMC encoding)

Tóm tắt một dòng: Code trong Lec 3 dùng làm **input để demo BMC encoding**. Mỗi sample là chương trình nhỏ có bug subtle mà BMC bắt được nhưng test ngẫu nhiên có thể miss. Phân tích lại để hiểu vì sao BMC quan trọng và cách encode chính xác.

Sample 1: Array bounds với index từ symbolic

Nguồn: Lec 3, mục State Space Exploration / BMC intro.

Code (cleaned up):

```
#include <assert.h>

int nondet_int(void);

int main(void) {
    int a[2], i, x;
    i = nondet_int();
    x = nondet_int();
    if (x == 0)
        a[i] = 0;
    else
        a[i] = 1;
    assert(a[i + 1] == 1);    // (1) BUG  $n \models i + 1 \geq 2$ 
    return 0;
}
```

Đọc nhanh: hai biến nondet i, x. Gán a[i] theo x. Assert a[i+1] == 1.

Bug tìm được:

1. **Bug chính (Critical):** i không bound. Nếu i == 0, a[1] được assert. Nhưng a[1] chưa khởi tạo (uninitialized) $\square$ undefined value, assert có thể fail.
2. **Bug phụ (Critical):** nếu i $\geq$ 1, a[i+1] OOB. Read OOB = UB.
3. **Bug phụ (High):** nếu i < 0, a[i] = ... OOB write. Memory corruption.

Cơ chế BMC bắt:

BMC encode chương trình thành SMT:

```
(declare-fun a () (Array Int Int))
(declare-fun i () Int)
(declare-fun x () Int)

; Path 1: x == 0
(assert (=> (= x 0) (= a_new (store a i 0))))
; Path 2: x != 0
(assert (=> (not (= x 0)) (= a_new (store a i 1))))

; Property: a_new[i+1] == 1
(assert (not (= (select a_new (+ i 1)) 1)))

(check-sat) ; SAT  $\rightarrow$  counterexample t $\models$  n tại
```

Z3 trả về $i = 0$, $x = 0$: assertion fail vì $a[1]$ không được gán (giá trị mặc định trong SMT array là arbitrary, có thể khác 1).

Fix gợi ý (nếu intent là test concept BMC, không phải production):

```
#include <assert.h>

int nondet_int(void);

int main(void) {
    int a[2] = {0, 0};    // kho i tạo
    int i = nondet_int();
    int x = nondet_int();

    // Constraint i trong range hợp lệ
    if (i < 0 || i >= 1) return 0; // cho test i = 0

    if (x == 0)
        a[i] = 0;
    else
        a[i] = 1;

    // Bây giờ a[i+1] luôn là 0 (chưa modify)
    assert(a[i + 1] == 0); // assertion đúng
    return 0;
}
```

Verify với CBMC:

```
cbmc bounds.c --bounds-check
```

Output cho code gốc:

```
[main.array_bounds.1] line 11 array `a' upper bound: FAILURE
```

Bài học: code dùng làm BMC harness phải **rõ ràng về range của input**. `nondet_int()` không bound = BMC explore mọi int 32-bit. Dùng `__CPROVER_assume(0 <= i && i < N)` để focus.

Sample 2: Pointer arithmetic và alias

Nguồn: Lec 3, mục Encoding Pointers and Memory.

Code:

```
#include <assert.h>

int nondet_int(void);

int main(void) {
    int a[2], i, x, *p;
    p = a;
    i = nondet_int();
```

```

x = nondet_int();

if (x == 0)
    a[i] = 0;
else
    a[i] = 1;

assert(*(p + 2) == 1);    // (1) BUG
return 0;
}

```

Độc nhanh: tương tự sample 1 nhưng truy cập qua pointer.

Bug:

1. **Bug chính (Critical):** $*(p + 2) = a[2]$, OOB (mảng size 2, valid index 0..1).
2. **Bug phụ:** $p = a$ gán không cần ép kiểu (OK), nhưng $*(p + i + 1)$ cũng OOB nếu $i \geq 1$.

Cơ chế encoding pointer trong BMC:

CBMC dùng “**pointer encoding**” gồm 2 field: - Object ID: identifier của allocated block. - Offset: byte offset trong block.

```

p = a:
  object_id(p) = object_id(a)
  offset(p) = 0

```

```

p + 2:
  object_id(p+2) = object_id(p)
  offset(p+2) = offset(p) + 2 * sizeof(int) = 8

```

```

*(p+2):
  Read at object_id=a, offset=8

```

Array a has size $2 * \text{sizeof(int)} = 8$.
Valid offset: 0..7.
Offset 8 = OOB!

CBMC bắt OOB qua check `offset < object_size`.

Fix: tương tự sample 1, hoặc tăng buffer size:

```

int a[3];    // đ□ ch□ cho a[2]

```

Nhưng nếu intent là demo BMC bắt OOB, code giữ nguyên là **input demo đúng**.

Verify:

```

cbmc pointer_arith.c --pointer-check --bounds-check

```

[main.pointer_dereference.1] line 12 dereference failure: object boundaries: FAILURE

Bài học: BMC encode pointer chính xác (object + offset). Tool detect mọi OOB qua pointer arithmetic, không chỉ array index syntax.

Sample 3: Floating-point encoding

Nguồn: Lec 3, mục Encoding Floats / IEEE 754.

Code:

```
#include <assert.h>
#include <math.h>

double nondet_double(void);

int main(void) {
    double a = 0.1;
    double b = 0.2;
    double c = a + b;
    assert(c == 0.3);    // (1) BUG: float không exact
    return 0;
}
```

Độc nhanh: kiểm tra $0.1 + 0.2 == 0.3$.

Bug:

1. **Bug chính (Logic):** 0.1 và 0.2 không biểu diễn được chính xác trong IEEE 754 double. $0.1 + 0.2 \approx 0.30000000000000004$, không bằng 0.3 (cũng $\approx 0.29999999999999998$).

Cơ chế:

IEEE 754 double dùng base 2 mantissa. Số 0.1 trong base 2 là tuần hoàn vô hạn: $0.0001100110011\dots$. Cần round $\square$ mất chính xác.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Bug này xuất hiện trong tài chính (lãi suất compound), khoa học (simulation), ML (gradient).

Fix tùy context:

Option A: compare với epsilon

```
#include <math.h>

#define EPS 1e-9

if (fabs(c - 0.3) < EPS) {
    // c "approximately equal" 0.3
}
```

Option B: dùng decimal arithmetic

C không có built-in decimal. Dùng library hoặc C++ `boost::multiprecision::cpp_dec_float`.

Option C: dùng integer (cents)

Tài chính nên dùng integer biểu diễn cents thay vì float biểu diễn dollars:

```
int64_t cents_a = 10;    // $0.10
int64_t cents_b = 20;    // $0.20
int64_t cents_c = cents_a + cents_b;    // 30 cents = $0.30, exact
```

Verify với CBMC:

```
cbmc float.c --float-overflow-check
```

CBMC dùng theory FP (IEEE 754), detect được assertion sai.

Bài học: KHÔNG dùng == cho float. KHÔNG dùng float cho money. KaBoom 1-2 sai sót trong domain critical.

Sample 4: Float comparison subtle

Nguồn: Lec 3, ví dụ float edge case.

Code:

```
#include <math.h>
#include <assert.h>

int main(void) {
    double x = 1.0 / 0.0;    // +Infinity
    double y = -1.0 / 0.0;   // -Infinity
    double n = 0.0 / 0.0;    // NaN

    assert(x > y);           // (1) OK: +Inf > -Inf
    assert(n != n);          // (2) Đúng! NaN != NaN
    assert(n == n);          // (3) BUG: this fails
    return 0;
}
```

Đọc nhanh: demo edge case IEEE 754 (+Inf, -Inf, NaN).

Bug:

1. Assertion 1 OK.
2. Assertion 2 đúng (NaN không equal anything, kể cả chính nó).
3. **Assertion 3 SAI:** `n == n` is false vì `n` là NaN. Assertion fail.

Cơ chế:

IEEE 754 đặc biệt: $-1.0 / 0.0 = +\infty$ (positive infinity), không raise exception trong default mode. $-0.0 / 0.0 = \text{NaN}$ (not a number). $-\text{NaN} \text{ op anything} = \text{NaN}$. $-\text{NaN} == \text{NaN} = \text{false}$ (theo design, để detect NaN qua `x != x`).

Fix (cách check NaN đúng):

```
#include <math.h>

if (isnan(n)) {
    // n là NaN
}
```

`isnan()` là macro chuẩn C99.

Bài học: code dùng float phải có NaN handling. CBMC `--nan-check` bắt operation tạo ra NaN.

Sample 5: SSA conversion example

Nguồn: Lec 3, mục SSA encoding (concept).

Code gốc:

```
int x = 0;
if (cond) {
    x = 1;
} else {
    x = 2;
}
y = x;
```

Đọc nhanh: gán x dựa trên cond, rồi assign y.

Đây không phải bug code, mà là **input để BMC convert sang SSA**.

SSA form:

```
int x_0 = 0;
int x_1, x_2;
if (cond) {
    x_1 = 1;
} else {
    x_2 = 2;
}
int x_3 = cond ? x_1 : x_2;    // phi-node
y_0 = x_3;
```

Mỗi assignment tạo version mới của biến. Tại merge point (sau if-else), phi-node chọn version đúng.

SMT encoding tương đương:

```
(declare-const cond Bool)
(declare-const x_1 Int)
(declare-const x_2 Int)
(declare-const x_3 Int)
(declare-const y_0 Int)

(assert (=> cond (= x_1 1)))
(assert (=> (not cond) (= x_2 2)))
(assert (= x_3 (ite cond x_1 x_2)))
(assert (= y_0 x_3))
```

Sample minh hoạ phi-node confusion:

```
int main(void) {
    int x = 0;
    if (cond) {
        x = 1;
    }
}
```



```

    // x_2 = phi(x_1, x_0)

    int y = x;
    // y == 0 hoặc 1, không xác định tại compile time

    assert(y == 0);    // BUG nh u cond = true
}

```

Verify với CBMC:

cbmc ssa.c

Output:

[main.assertion.1] line 8 assertion y == 0: FAILURE
Counterexample: cond = true → y = 1, assertion fail

Bài học: SSA là **internal representation** của compiler/BMC, không phải bug. Hiểu SSA giúp đọc CBMC output có “x_1, x_2, x_3” trong counterexample.

Sample 6: Array assignment với index nondet

Nguồn: Lec 3, mục Array theory.

Code:

```

#include <assert.h>

int nondet_int(void);

int main(void) {
    int a[10];
    int i = nondet_int();
    int v = nondet_int();

    if (i >= 0 && i < 10) {
        a[i] = v;
        assert(a[i] == v);    // (1) OK?
    }
    return 0;
}

```

Đọc nhanh: nondet i và v, gán a[i] = v, assert a[i] = v.

Phân tích: assertion **luôn đúng** (write-then-read same index = giá trị vừa ghi).

Nhưng nếu thay đổi nhẹ:

```

a[i] = v;
a[j] = v + 1;    // j cũng nondet
assert(a[i] == v);    // (2) BUG nh u i == j

```

Khi i == j, a[j] = v+1 đè lên giá trị vừa ghi. a[i] giờ là v+1, assertion fail.

Cơ chế array theory:

SMT array theory với axiom: - $\text{select}(\text{store}(a, i, v), j) = v$ nếu $i == j$. - $\text{select}(\text{store}(a, i, v), j) = \text{select}(a, j)$ nếu $i != j$.

BMC apply axiom này khi solve, bắt được trường hợp $i == j$.

Fix:

```
if (i != j) {
    a[i] = v;
    a[j] = v + 1;
    assert(a[i] == v);    // OK
}
```

Verify:

cbmc array_alias.c

[main.assertion.1] line 11 assertion a[i] == v: FAILURE

Counterexample: i = j = 5, v = 0

Bài học: array alias (hai index có thể equal) là pattern subtle. BMC handle đúng nhờ array theory.

Sample 7: Memory model với multiple pointer

Nguồn: Lec 3, mục Encoding pointers and memory.

Code:

```
#include <assert.h>

int main(void) {
    int x = 10;
    int *p = &x;
    int *q = &x;

    *p = 20;
    assert(*q == 20);    // (1) OK vì p và q alias

    *q = 30;
    assert(x == 30);    // (2) OK
    return 0;
}
```

Độc nhanh: hai pointer cùng trỏ x, mutate qua p, read qua q.

Phân tích: assertion đều đúng nếu compiler không optimize sai (volatile-like).

Bug subtle:

```
int x = 10;
int *p = &x;
int *q = (int*)((char*)&x + 1);    // misaligned pointer
*p = 0;
*q = 1;
assert(x == 0);    // Behavior phụ thuộc memory layout
```

q không aligned cho int. Behavior **undefined**. Trên x86 thường work nhưng slow, trên ARM strict alignment crash.

Cơ chế memory model:

CBMC có 3 memory model: - `--mm fixed`: object có fixed address. - `--mm align`: object align theo type. - `--mm offset`: track offset chính xác trong object.

Default là align, đủ cho hầu hết code C đúng.

Bài học: pointer alias đúng (cùng object, aligned) OK. Pointer **misaligned hoặc cross-object** là UB. CBMC bắt qua `--pointer-check`.

Sample 8: Encoding integer division

Nguồn: Lec 3, mục Encoding numbers.

Code:

```
#include <assert.h>
#include <limits.h>

int nondet_int(void);

int main(void) {
    int a = nondet_int();
    int b = nondet_int();

    int c = a / b;    // (1) BUG n ÷ u b == 0
    int d = a % b;    // (2) BUG n ÷ u b == 0

    int e = INT_MIN / -1;    // (3) BUG: signed overflow!

    assert(c * b + d == a);    // (4) Math identity: thường đúng
    return 0;
}
```

Bug:

1. **Bug 1 (Critical)**: division by zero là UB.
2. **Bug 2 (Critical)**: modulo by zero là UB.
3. **Bug 3 (High)**: `INT_MIN / -1` overflow vì `|INT_MIN| > INT_MAX`. Crash trên x86 (FPE).
4. Identity thường đúng nhưng có thể fail nếu b âm với mod convention khác (C99 uses truncated division).

Cơ chế CBMC:

```
cbmc div.c --div-by-zero-check --signed-overflow-check
```

```
[main.division-by-zero.1] line 8 division by zero: FAILURE
[main.division-by-zero.2] line 9 modulo by zero: FAILURE
[main.overflow.1] line 11 arithmetic overflow on signed division: FAILURE
```

CBMC bắt cả 3 bug.

Fix:

```

if (b == 0) return -1;
if (a == INT_MIN && b == -1) return -1; // overflow case

int c = a / b;
int d = a % b;

```

Bài học: division by zero và INT_MIN / -1 là 2 case ít người nhớ. BMC bắt cả 2.

Tóm tắt Cụm 2

| Sample | Lớp bug | Tool detect |
|-----------------------|-----------|------------------------------|
| 1 Array bounds nondet | OOB | CBMC --bounds-check |
| 2 Pointer arith OOB | OOB | CBMC --pointer-check |
| 3 Float compare | Logic | CBMC --float-overflow-check |
| 4 NaN handling | Logic | CBMC --nan-check |
| 5 SSA / phi-node | (concept) | CBMC trace |
| 6 Array alias | Logic | CBMC array theory |
| 7 Pointer alias | Memory | CBMC --pointer-check |
| 8 Division overflow | Integer | CBMC --signed-overflow-check |

8 sample này demo **mọi loại check** mà BMC tool cung cấp built-in.

Pattern lập harness BMC tốt

Sau khi đọc 8 sample, có thể đúc kết **best practice viết harness cho BMC**:

1. **Khởi tạo rõ ràng:** `int a[10] = {0}` thay vì uninitialized.
2. **Bound input:** `__CPROVER_assume(0 <= i && i < N)` để focus.
3. **Một assertion / harness:** dễ debug counterexample.
4. **Enable mọi check:** `--bounds-check --pointer-check --signed-overflow-check ...`
5. **Set unwind đủ lớn:** loop bound > thực tế ít nhất 2x.
6. **Lưu seed:** nếu counterexample đẹp, lưu làm regression test.

Code production thường khó verify trực tiếp. Tạo “**verification harness**” riêng, gọi target function với input nondet bounded.

Tiếp theo: 8.4 Code patterns Cụm 3 (concurrency)

8.4 Code patterns Cúm 3 (Lec 4 - Concurrency)

Tóm tắt một dòng: Code Lec 4 là các program đa luồng minh họa class lỗi concurrency: data race, atomicity violation, deadlock, memory model. Khó debug bằng test thông thường vì bug **non-deterministic**, nhưng CBMC --pthread khám phá interleaving exhaustively trong bound.

Sample 1: Race trên shared variable

Nguồn: Lec 4, mục Concurrency Verification.

Code (cleaned up):

```
#include <pthread.h>
#include <assert.h>

int n = 0;

void* P(void *arg) {
    n++;
    return NULL;
}

int main(void) {
    pthread_t id1, id2;
    pthread_create(&id1, NULL, P, NULL);
    pthread_create(&id2, NULL, P, NULL);
    pthread_join(id1, NULL);
    pthread_join(id2, NULL);
    assert(n == 2); // (1) BUG?
    return 0;
}
```

Độc nhanh: 2 thread mỗi cái n++ 1 lần. Expect n == 2.

Bug:

1. **Bug (High):** n++ không atomic. Race possible, n có thể = 1 (lost update).

Cơ chế interleaving:

n++ = 3 instruction: LOAD r, n; INC r; STORE n, r.

Schedule “bad”:

```
T1: LOAD r1 from n (r1 = 0)
T2: LOAD r2 from n (r2 = 0)
T1: INC r1 (r1 = 1)
T2: INC r2 (r2 = 1)
T1: STORE r1 to n (n = 1)
T2: STORE r2 to n (n = 1)
Final: n = 1, NOT 2
```

Schedule “good”:

T1: LOAD, INC, STORE → n = 1
T2: LOAD, INC, STORE → n = 2
Final: n = 2

Probability của bad schedule trên modern CPU: phụ thuộc memory model, contention. Test 1 triệu lần có thể vẫn không hit.

Verify với CBMC:

```
cbmc race.c --pthread --unwind 5
```

CBMC khám phá mọi interleaving khả dĩ. Output:

[main.assertion.1] line 17 assertion n == 2: FAILURE

Schedule:

T1: LOAD n=0
T2: LOAD n=0
T1: INC, STORE n=1
T2: INC, STORE n=1
→ n = 1

Fix với atomic:

```
#include <stdatomic.h>

atomic_int n = 0;

void* P(void *arg) {
    atomic_fetch_add(&n, 1);
    return NULL;
}
```

Fix với mutex:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int n = 0;

void* P(void *arg) {
    pthread_mutex_lock(&lock);
    n++;
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

Bài học: mọi shared variable phải hoặc atomic, hoặc protected by lock. Single instruction nguồn gốc thường không atomic.

Sample 2: Atomicity violation

Nguồn: Lec 4, mục Atomicity Violation.

Code:

```

#include <pthread.h>

int balance = 100;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void withdraw(int amount) {
    pthread_mutex_lock(&lock);
    if (balance >= amount) {
        // (1) lock release before action
        pthread_mutex_unlock(&lock);
        // Tại đây thread khác có thể withdraw
        balance -= amount; // (2) BUG: race
    } else {
        pthread_mutex_unlock(&lock);
    }
}

```

Bug:

1. **Bug (High):** lock được giữ trong check nhưng release trước modify. Hai thread cùng pass check, cùng subtract $\Rightarrow$ balance âm.

Cơ chế:

```

balance = 100
T1: lock, check (100 >= 60: true), unlock
T2: lock, check (100 >= 60: true), unlock
T1: balance -= 60  $\rightarrow$  balance = 40
T2: balance -= 60  $\rightarrow$  balance = -20 (NEGATIVE!)

```

Fix:

```

void withdraw(int amount) {
    pthread_mutex_lock(&lock);
    if (balance >= amount) {
        balance -= amount; // (3) inside critical section
    }
    pthread_mutex_unlock(&lock);
}

```

Quy tắc: **check và action phải trong cùng critical section.**

Verify với CBMC:

```
cbmc withdraw.c --pthread --unwind 3
```

CBMC tìm schedule khiến $\text{balance} < 0$.

Bài học: lock granularity quan trọng. Lock quá fine = bug atomicity. Lock quá coarse = giảm performance.

Sample 3: Deadlock cổ điển

Nguồn: Lec 4, mục Deadlock.

Code:

```

#include <pthread.h>

pthread_mutex_t lockA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lockB = PTHREAD_MUTEX_INITIALIZER;

void* T1(void *arg) {
    pthread_mutex_lock(&lockA);
    // ... work ...
    pthread_mutex_lock(&lockB);    // (1)
    // ... work ...
    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);
    return NULL;
}

void* T2(void *arg) {
    pthread_mutex_lock(&lockB);
    // ... work ...
    pthread_mutex_lock(&lockA);    // (2)
    // ... work ...
    pthread_mutex_unlock(&lockA);
    pthread_mutex_unlock(&lockB);
    return NULL;
}

```

Bug:

1. **Bug (Critical):** T1 lock A trước, T2 lock B trước. Schedule:

T1: lock A T2: lock B
 T1: WAIT for B T2: WAIT for A
 DEADLOCK

Cơ chế: bốn điều kiện deadlock (Coffman 1971): 1. Mutual exclusion. 2. Hold and wait. 3. No preemption. 4. Circular wait.

Code trên thoả mọi điều kiện $\square$ deadlock guaranteed under right schedule.

Fix (lock ordering):

```

// Mọi thread acquire lock theo cùng thứ tự
void* T1(void *arg) {
    pthread_mutex_lock(&lockA);    // luôn A trước
    pthread_mutex_lock(&lockB);
    // ...
    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);
    return NULL;
}

void* T2(void *arg) {
    pthread_mutex_lock(&lockA);    // A trước B

```



```

pthread_mutex_lock(&lockB);
// ...
pthread_mutex_unlock(&lockB);
pthread_mutex_unlock(&lockA);
return NULL;
}

```

Verify với CBMC:

```
cbmc deadlock.c --pthread --deadlock-check --unwind 5
```

CBMC bắt circular wait.

Bài học: lock ordering convention global. Define order once (e.g. by address), enforce ở code review. Tool tsan cũng detect deadlock runtime.

Sample 4: Lost wakeup

Nguồn: Lec 4, mục condition variable.

Code:

```

#include <pthread.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
int ready = 0;

void* consumer(void *arg) {
    pthread_mutex_lock(&lock);
    if (!ready) { // (1) BUG: should be while
        pthread_cond_wait(&cv, &lock);
    }
    // ... use shared data ...
    pthread_mutex_unlock(&lock);
    return NULL;
}

void* producer(void *arg) {
    pthread_mutex_lock(&lock);
    ready = 1;
    pthread_cond_signal(&cv); // (2) hoặc broadcast
    pthread_mutex_unlock(&lock);
    return NULL;
}

```

Bug:

1. **Bug (Medium):** if (!ready) thay vì while (!ready). Spurious wakeup hoặc multiple consumer = bug.
2. **Bug phụ:** pthread_cond_signal chỉ wake 1 thread. Nhiều consumer waiting = lost wakeup.

Cơ chế spurious wakeup:

POSIX cho phép `pthread_cond_wait` return ngẫu nhiên (kernel optimization). Nếu code `if`, sau wake thread tiếp tục mà không recheck condition ☐ bug.

Fix chuẩn:

```
void* consumer(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) {           // ĐÚNG: while, không if
        pthread_cond_wait(&cv, &lock);
    }
    // ... use shared data ...
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

`pthread_cond_broadcast` nếu có nhiều consumer cần wake.

Verify:

CBMC --pthread model `pthread_cond_wait` chính xác với spurious wakeup.

Bài học: pattern “condition wait” luôn dùng `while`, không bao giờ `if`. Quy tắc 1 dòng nhưng nhiều người sai.

Sample 5: Memory model (TSO surprise)

Nguồn: Lec 4, mục Memory Model.

Code (Dekker’s algorithm simplified):

```
#include <pthread.h>
#include <assert.h>

int x = 0, y = 0;
int r1, r2;

void* T1(void *arg) {
    x = 1;
    r1 = y;    // read y
    return NULL;
}

void* T2(void *arg) {
    y = 1;
    r2 = x;    // read x
    return NULL;
}

int main(void) {
    pthread_t id1, id2;
    pthread_create(&id1, NULL, T1, NULL);
    pthread_create(&id2, NULL, T2, NULL);
    pthread_join(id1, NULL);
```

```
pthread_join(id2, NULL);

// Under Sequential Consistency: r1 == 1 || r2 == 1 always
assert(r1 == 1 || r2 == 1); // (1) BUG under TSO
return 0;
}
```

Đọc nhanh: T1 write x then read y. T2 write y then read x. Property: ít nhất 1 read thấy “1”.

Bug under TSO (x86 memory model):

CPU có **store buffer**: write x = 1 vào buffer, không immediately commit. Read r1 = y có thể happen trước commit. Tương tự T2.

Schedule possible under TSO (impossible under SC):

T1: write x=1 to store buffer (delayed)
T2: write y=1 to store buffer (delayed)
T1: read y from memory → r1 = 0 (chưa thấy y y=1)
T2: read x from memory → r2 = 0 (chưa thấy x x=1)
Later: store buffer flush
Final: r1 = 0, r2 = 0. Assertion FAIL!

Cơ chế: TSO cho phép **store-load reordering**. x86 dùng TSO (chính xác là x86-TSO). ARM/POWER yếu hơn, cho phép thêm reordering.

Fix với memory fence:

```
#include <stdatomic.h>

void* T1(void *arg) {
    x = 1;
    atomic_thread_fence(memory_order_seq_cst); // full fence
    r1 = y;
    return NULL;
}

void* T2(void *arg) {
    y = 1;
    atomic_thread_fence(memory_order_seq_cst);
    r2 = x;
    return NULL;
}
```

Fence force CPU flush store buffer trước read.

Fix với atomic + seq_cst:

```
atomic_int x = 0, y = 0;

void* T1(void *arg) {
    atomic_store(&x, 1);
    r1 = atomic_load(&y);
    return NULL;
}
```

```
}
```

Default atomic_store/load dùng memory_order_seq_cst, đảm bảo SC.

Verify với CBMC:

```
cbmc dekker.c --pthread --mm tso --unwind 3
```

CBMC support multiple memory model. Default SC. --mm tso test under TSO.

Bài học: code lock-free đòi hỏi hiểu memory model. Tránh trừ khi cần performance cực cao. Dùng mutex hoặc seq_cst atomic cho 99% case.

Sample 6: ABA problem trong lock-free

Nguồn: Lec 4, ABA pattern.

Code (stack lock-free naïve):

```
#include <stdatomic.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

atomic_uintptr_t top; // pointer to head

void push(Node *n) {
    Node *old_top;
    do {
        old_top = (Node*)atomic_load(&top);
        n->next = old_top;
    } while (!atomic_compare_exchange_weak(&top, (uintptr_t*)&old_top, (uintptr_t)n));
}

Node* pop(void) {
    Node *old_top, *new_top;
    do {
        old_top = (Node*)atomic_load(&top);
        if (old_top == NULL) return NULL;
        new_top = old_top->next;
        // (1) ABA window
    } while (!atomic_compare_exchange_weak(&top, (uintptr_t*)&old_top, (uintptr_t)new_top));
    return old_top;
}
```

Bug:

1. **Bug (High) - ABA:** giữa load old_top và CAS, một thread khác có thể pop, free, push back same address. CAS thấy “vẫn old_top” nhưng nội dung đã khác.

Cơ chế ABA:

Initial: stack = A → B → C

T1: pop, load top = A, next = B → about to CAS

T2: pop A (returns A)

T2: pop B (returns B)

T2: push A' at SAME ADDRESS as A

T1: CAS succeeds (A == A by pointer), but next is now C, not B

Bug: B is leaked, list broken

Fix 1 (hazard pointer): complex, dùng trong production lock-free.

Fix 2 (tag pointer): dùng 2 phần (pointer + version counter):

```
typedef struct {
    Node *ptr;
    uint64_t version;
} TaggedPtr;
```

atomic of TaggedPtr (need 128-bit atomic on x86_64).

Mỗi modify increment version. CAS compare cả ptr và version.

Fix 3 (đơn giản nhất): dùng mutex, không lock-free.

Verify:

CBMC --pthread có thể model lock-free với bounded exploration, nhưng ABA cần model memory allocator chính xác.

Bài học: lock-free data structure cực kỳ khó đúng. ABA chỉ là một trong nhiều bug class. Nếu không có lý do performance critical, **dùng mutex**.

Sample 7: Condition race trong initialization

Nguồn: Lec 4, double-checked locking.

Code (double-checked locking):

```
#include <pthread.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
Singleton *instance = NULL;

Singleton* get_instance(void) {
    if (instance == NULL) {                                // (1) check without lock
        pthread_mutex_lock(&lock);
        if (instance == NULL) {                            // (2) double check
            instance = create_singleton();                 // (3) BUG under weak mem
        }
        pthread_mutex_unlock(&lock);
    }
    return instance;
}
```

Bug:

1. **Bug (Medium-High) on weak memory:** `instance = create_singleton()` không atomic.
Compiler có thể reorder:
 - First write pointer.
 - Then initialize object fields.

Reader thread thấy `instance != NULL` (qua check at (1)), return pointer, dereference fields chưa init
□ garbage hoặc crash.

Cơ chế:

T1: alloc memory at address X
T1: write `instance = X` ← reader can see this
T1: initialize *X
T2: check `instance`, see X (not NULL)
T2: dereference X → fields chưa init

Trên x86 (TSO) thường OK vì store ordered. Trên ARM/POWER (weaker), bug rõ.

Fix với atomic + release/acquire:

```
#include <stdatomic.h>

atomic_uintptr_t instance = ATOMIC_VAR_INIT(0);

Singleton* get_instance(void) {
    Singleton *p = (Singleton*)atomic_load_explicit(&instance, memory_order_acquire);
    if (p == NULL) {
        pthread_mutex_lock(&lock);
        p = (Singleton*)atomic_load_explicit(&instance, memory_order_acquire);
        if (p == NULL) {
            p = create_singleton();
            atomic_store_explicit(&instance, (uintptr_t)p, memory_order_release);
        }
        pthread_mutex_unlock(&lock);
    }
    return p;
}
```

`memory_order_release` (writer) + `memory_order_acquire` (reader) đảm bảo: nếu reader thấy pointer mới, mọi write trước nó đều visible.

Fix tốt nhất: dùng `pthread_once` hoặc C++ `std::call_once`.

```
pthread_once_t init_done = PTHREAD_ONCE_INIT;

void init(void) {
    instance = create_singleton();
}

Singleton* get_instance(void) {
    pthread_once(&init_done, init);
    return instance;
}
```

Pattern này standard, đảm bảo init đúng 1 lần thread-safe.

Bài học: double-checked locking là pattern **infamous khó đúng**. Dùng `call_once` / `pthread_once` thay thế.

Sample 8: False sharing performance bug

Nguồn: Lec 4 (extension), cache line.

Code:

```
#include <pthread.h>

struct Counters {
    int a;
    int b;
};

struct Counters counters;

void* T1(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        counters.a++; // (1) atomic? hoặc dùng atomic_int
    }
    return NULL;
}

void* T2(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        counters.b++;
    }
    return NULL;
}
```

Bug:

1. Functional bug: cả 2 đều race (sample 1). Giả sử fix bằng atomic.
2. **Performance bug (Medium): false sharing.** a và b nằm cùng cache line 64 byte. Mọi modify cache line invalidate cho thread khác $\square$ cache miss $\square$ slow.

Cơ chế:

CPU cache hoạt động ở granularity 64-byte line. `int a`, `int b` (8 byte) nằm cùng 1 line. T1 modify a $\square$ invalidate line cho T2. T2 modify b $\square$ invalidate line cho T1. Ping-pong cache, không có race functional nhưng performance giảm 10-100x.

Fix với cache line padding:

```
#include <stdatomic.h>

struct Counters {
    alignas(64) atomic_int a;
    alignas(64) atomic_int b;
};
```

`alignas(64)` đảm bảo a và b ở khác cache line.

Hoặc dùng padding manual:

```
struct Counters {
    atomic_int a;
    char pad1[60];    // pad to 64 bytes
    atomic_int b;
    char pad2[60];
};
```

Verify: false sharing không phải functional bug, BMC không detect. Dùng `perf c2c` (Linux) hoặc Intel VTune để measure cache miss.

Bài học: false sharing là performance bug subtle. Pattern: thread-private counter padding tránh sharing. Quan trọng cho high-throughput data path.

Tóm tắt Cụm 3

| Sample | Bug | Tool detect |
|-----------------------|---------------|------------------------------|
| 1 Race n++ | Data race | CBMC –pthread, TSan |
| 2 Atomicity violation | Logic race | CBMC –pthread |
| 3 Deadlock | Lock ordering | CBMC –deadlock-check, TSan |
| 4 Lost wakeup | Cond var | CBMC –pthread |
| 5 TSO reordering | Memory model | CBMC –mm tso |
| 6 ABA lock-free | Memory reuse | Manual review (BMC limited) |
| 7 Double-check init | Memory model | CBMC –pthread + memory order |
| 8 False sharing | Performance | perf c2c, VTune |

8 sample này demo **toàn bộ class concurrency bug** trong thực tế.

Pattern an toàn cho concurrency

Sau khi đọc, các quy tắc cốt lõi:

1. **Mọi shared variable: atomic hoặc protected by lock.** Không có “I’ll just be careful”.
2. **Lock ordering global:** tránh circular deadlock.
3. **Condition wait luôn dùng while:** chống spurious wakeup.
4. **Atomic mặc định seq_cst:** weaker order chỉ khi đo performance critical.
5. **Tránh lock-free trừ khi cần:** ABA, memory model, leak khó debug.
6. **pthread_once cho lazy init:** thay double-checked locking.
7. **Cache padding cho hot counter:** tránh false sharing.

7 quy tắc này chống 90% concurrency bug thực tế. Còn 10% còn lại là expert territory (kernel, allocator, JIT runtime).

Tiếp theo: 8.5 Code patterns Cụm 4 (testing và fuzzing)

8.5 Code patterns Cúm 4 (Lec 5 - Testing và Fuzzing)

Tóm tắt một dòng: Code Lec 5 chia 3 nhóm: (1) program nhỏ minh hoạ coverage criteria (statement, branch, MC/DC), (2) fuzz target trông như production function nhưng có bug subtle, (3) symbolic execution example. Bài này đọc lại từng nhóm, comment chi tiết, và viết fuzz harness chuẩn.

Sample 1: Coverage demo program

Nguồn: Lec 5, mục Coverage Criteria (statement coverage).

Code:

```
#include "lib.h"

int max_or_default(int a, int b, int def) {
    int result;
    if (a > b) {
        result = a;
    } else if (b > a) {
        result = b;
    } else {
        result = def;
    }
    return result;
}
```

Đọc nhanh: trả về max của a và b, hoặc default nếu equal.

Phân tích coverage:

Statement coverage 100%: cần test cover mọi statement. - Test 1: a=5, b=3, def=0 → vào nhánh a > b, result = 5. Cover line result = a. - Test 2: a=3, b=5, def=0 → nhánh b > a, result = 5. Cover line result = b. - Test 3: a=5, b=5, def=99 → nhánh else, result = 99. Cover line result = def.

3 test cover hết statement.

Branch coverage 100%: cần cover mọi branch true/false. - a > b true: test 1. False: test 2 và 3. - b > a true: test 2. False: test 3.

3 test trên cover hết branch.

Bug subtle (có thể không bị detect bởi 3 test trên):

- **Bug 1:** nếu a == INT_MIN, b == INT_MAX: a > b false, b > a true, return INT_MAX. OK.
- **Bug 2:** nếu a == INT_MAX, b == INT_MIN: a > b true, return INT_MAX. OK.

Function này thực ra **đúng**. Đây là test case “happy path”.

Test edge case bằng CBMC:

```
#include <assert.h>
#include <limits.h>

int max_or_default(int a, int b, int def);
```

```

int main(void) {
    int a = nondet_int();
    int b = nondet_int();
    int def = nondet_int();

    int result = max_or_default(a, b, def);

    // Property: result is max(a, b) hoặc def n u equal
    if (a > b) assert(result == a);
    else if (b > a) assert(result == b);
    else assert(result == def);
    return 0;
}

```

cbmc harness.c lib.c

Nếu function đúng, CBMC chứng minh assertion hold cho **mọi input 32-bit**.

Bài học: 100% coverage **không đảm bảo bug-free**. Nhưng < 100% coverage = có code không test, chắc chắn có rủi ro. Coverage là **necessary but not sufficient**.

Sample 2: Code có MC/DC challenge

Nguồn: Lec 5, mục MC/DC coverage.

Code:

```

int check_access(int is_admin, int is_owner, int file_public) {
    if ((is_admin || is_owner) && !file_public) {
        return 1; // access granted
    }
    return 0;
}

```

Wait - logic này sai? Let me re-read.

Actually let me reverse it - it makes more sense:

```

int check_access(int is_admin, int is_owner, int file_public) {
    if ((is_admin || is_owner) || file_public) {
        return 1; // access granted
    }
    return 0;
}

```

Phân tích MC/DC:

Decision: (A || B) || C với A=is_admin, B=is_owner, C=file_public.

MC/DC yêu cầu: mỗi condition độc lập ảnh hưởng outcome.

Bảng truth: | Test | A | B | C | (A||B) | Result | |---|---|---|---|---|---| T1 | 0 | 0 | 0 | 0 | 0 | 0 | T2 | 1 | 1 | 0 | 0 | 1 | 1 |
| T3 | 0 | 1 | 1 | 0 | 1 | 1 | T4 | 0 | 0 | 1 | 1 | 0 | 1 | T5 | 1 | 1 | 1 | 0 | 1 | 1 | T6 | 1 | 1 | 0 | 1 | 1 | 1 | T7 | 0 | 1 | 1 | 1 | 1 | 1 |
| T8 | 1 | 1 | 1 | 1 | 1 | 1 |

Cần show mỗi condition độc lập:

- **A độc lập:** B và C giữ, A đổi outcome. So sánh T1 (A=0, out=0) với T2 (A=1, out=1). B=0, C=0. **OK.**
- **B độc lập:** A và C giữ, B đổi outcome. T1 (B=0, out=0) vs T3 (B=1, out=1). A=0, C=0. **OK.**
- **C độc lập:** A và B giữ, C đổi outcome. T1 (C=0, out=0) vs T4 (C=1, out=1). A=0, B=0. **OK.**

Test set MC/DC: T1, T2, T3, T4. 4 test cho 3 condition. Đúng $n + 1$.

Bug subtle:

Code này có **logic bug**: “OR với file_public” = bất kỳ ai cũng truy cập file public, bất kể admin hay owner. Có thể đúng intent, nhưng nếu intent là “AND not file_public” (private file, chỉ admin/owner) thì sai.

Fix nếu intent là “private”:

```
int check_access(int is_admin, int is_owner, int file_public) {
    if ((is_admin || is_owner) && !file_public) {
        return 1;
    }
    return 0;
}
```

MC/DC test set cho version này **khác hoàn toàn**. Phân tích lại MC/DC là bài tập cho người đọc.

Bài học: MC/DC catch logic interaction. Boolean expression phức tạp cần MC/DC + manual review intent.

Sample 3: Fuzz target naïve

Nguồn: Lec 5, mục Fuzzing Basics.

Code (fuzz target):

```
#include <stdint.h>
#include <stddef.h>

extern int parse_packet(const uint8_t *data, size_t size);

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    if (size > 0) {
        parse_packet(data, size);    // (1) just call, no oracle
    }
    return 0;
}
```

Đọc nhanh: fuzz harness chuẩn libFuzzer, gọi parse_packet với input random.

Vấn đề (không phải bug, mà thiếu):

1. **Không có oracle:** fuzzer chỉ detect crash, không detect “wrong output”. Nếu parser trả về sai data nhưng không crash, fuzzer miss.
2. **Không validate state:** sau parse, không check internal invariant.

Enhancement:

```
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    if (size < 2) return 0;
```

```

PacketState state;
int result = parse_packet(data, size, &state);

if (result == 0) { // parse succeeded
    // Property 1: parsed size <= input size
    assert(state.parsed_bytes <= size);

    // Property 2: round-trip
    uint8_t buf[1024];
    size_t serialized = serialize_packet(&state, buf, sizeof(buf));
    assert(serialized <= sizeof(buf));

    PacketState state2;
    int reparse = parse_packet(buf, serialized, &state2);
    assert(reparse == 0);
    assert(packet_state_equal(&state, &state2));
}
return 0;
}

```

Differential fuzzing: 2 implementation so sánh:

```

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    PacketState s1, s2;
    int r1 = parse_packet_v1(data, size, &s1);
    int r2 = parse_packet_v2(data, size, &s2);

    // Property: both parser agree on success/fail
    assert((r1 == 0) == (r2 == 0));
    if (r1 == 0 && r2 == 0) {
        assert(packet_state_equal(&s1, &s2));
    }
    return 0;
}

```

Differential fuzz tìm divergence giữa implementations. Hữu ích khi có reference (e.g. fast C version vs slow Python version).

Bài học: fuzz target tốt = (1) drive code path, (2) có property check, (3) idempotent.

Sample 4: Symbolic execution example

Nguồn: Lec 5, mục White-box Fuzzing.

Code (target cho DSE):

```

int check_input(int x, int y) {
    if (x > 100) {
        if (y < 50) {
            if (x + y > 200) {
                assert(0); // (1) BUG path
            }
        }
    }
}

```

```

    }
    }
    return 0;
}

```

Đọc nhanh: 3 nested if, tới được `assert(0)` chỉ nếu thoả 3 điều kiện.

Tính khả thi của path random:

- $x > 100$: với x random 32-bit, $P(x > 100) \approx 50\%$ (chỉ cần signed, half values).
- $y < 50$: $P(y < 50) \approx 50\%$.
- $x + y > 200$: phụ thuộc x, y .

$P(\text{reach assert with random}) \approx 12\text{-}25\%$. Random fuzz có thể trigger.

Nhưng nếu thay condition thành **magic constant**:

```

if (x == 0xDEADBEEF) {
    if (y == 0xCAFEFABE) {
        assert(0);
    }
}

```

$P(\text{random hit}) = 2^{-64}$. Random fuzz không bao giờ trigger (cần $\sim 5 \times 10^{17}$ year).

DSE approach:

DSE track symbolic constraint: - Path 1: $x == 0xDEADBEEF$ and $y == 0xCAFEFABE$. - Solve với Z3: instantly trả về $x = 0xDEADBEEF, y = 0xCAFEFABE$.

DSE tìm input chính xác trong giây.

Verify với CBMC (BMC for test gen):

```
cbmc check.c --cover decision
```

Hoặc explicit:

```

int main(void) {
    int x = nondet_int();
    int y = nondet_int();
    check_input(x, y);
    return 0;
}

```

```
cbmc check.c --trace
```

CBMC trả về counterexample: $x = 0xDEADBEEF, y = 0xCAFEFABE$.

Bài học: DSE/BMC **mạnh hơn random fuzzing** cho path có magic constant hoặc complex constraint. Combine = best (Driller pattern, đã thấy bài 5.7).

Sample 5: BMC for test generation

Nguồn: Lec 5, mục BMC for Test Gen.

Code (target function):

```
int classify(int x) {
    if (x < 0) return -1;
    if (x == 0) return 0;
    if (x < 10) return 1;
    if (x < 100) return 2;
    return 3;
}
```

Goal: sinh test set cover mọi return path.

Harness:

```
#include <assert.h>

int classify(int x);

void cover_path_neg(void) {
    int x = nondet_int();
    assert(classify(x) != -1);    // assert FAIL → counterexample là test cho path -1
}

void cover_path_zero(void) {
    int x = nondet_int();
    assert(classify(x) != 0);
}

void cover_path_small(void) {
    int x = nondet_int();
    assert(classify(x) != 1);
}

void cover_path_medium(void) {
    int x = nondet_int();
    assert(classify(x) != 2);
}

void cover_path_large(void) {
    int x = nondet_int();
    assert(classify(x) != 3);
}

int main(void) {
    cover_path_neg();
    return 0;
}
```

Chạy CBMC mỗi cover_path_X riêng:

```
cbmc classify.c --function cover_path_neg --trace
```

Output: counterexample x = -5 (e.g.). Đây là test case.

Tự động hoá: script generate harness cho mỗi path, chạy CBMC, collect counterexample □ test suite.

Tool có sẵn: cbmc --cover line sinh test cho mỗi line, --cover branch cho mỗi branch.

```
cbmc classify.c --cover branch
```

CBMC liệt kê branch chưa cover và sinh input cover được.

Bài học: BMC for test gen = “**negate goal, ask for counterexample**”. Đảo property thành assertion sai, BMC sinh input đạt path.

Sample 6: Fuzz harness cho parser

Nguồn: Lec 5, ví dụ fuzz parser binary format.

Code (target):

```
#include <stdint.h>
#include <string.h>

typedef struct {
    uint16_t magic;
    uint32_t version;
    uint32_t payload_len;
    uint8_t payload[];
} Packet;

int parse_packet(const uint8_t *data, size_t size, Packet *out) {
    if (size < 10) return -1; // header size = 10

    out->magic = (data[0] << 8) | data[1];
    if (out->magic != 0xCAFE) return -1;

    memcpy(&out->version, data + 2, 4);
    memcpy(&out->payload_len, data + 6, 4);

    // (1) BUG: integer overflow check missing
    if (size < 10 + out->payload_len) return -1;

    // (2) BUG: payload too big for output buffer
    memcpy(out->payload, data + 10, out->payload_len);
    return 0;
}
```

Bug:

1. **Bug (Critical):** out->payload_len is uint32_t. Attacker set payload_len = 0xFFFFFFFF. 10 + 0xFFFFFFFF overflow uint32 □ 8 (very small). Check passes!
2. **Bug (Critical):** assuming out->payload có đủ chỗ chứa payload_len byte. Nếu out là Packet packet (stack alloc nhỏ), overflow stack.

Fix:

```

int parse_packet(const uint8_t *data, size_t size, Packet *out, size_t out_max_payload) {
    if (size < 10) return -1;

    out->magic = (data[0] << 8) | data[1];
    if (out->magic != 0xCAFE) return -1;

    memcpy(&out->version, data + 2, 4);
    memcpy(&out->payload_len, data + 6, 4);

    // Check 1: payload_len trong giới hạn caller cho phép
    if (out->payload_len > out_max_payload) return -1;

    // Check 2: input đủ data cho payload_len declared
    // Dùng so sánh không overflow (size - 10 thay vì 10 + payload_len)
    if (size < 10 || size - 10 < out->payload_len) return -1;

    memcpy(out->payload, data + 10, out->payload_len);
    return 0;
}

```

Quy tắc: với check $a + b > c$, viết lại thành $a > c - b$ (nếu $c \geq b$) để tránh overflow.

Fuzz harness:

```

#include <stdint.h>
#include <stddef.h>
#include <stdlib.h>

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    if (size > 100000) return 0; // limit input size để fuzzer fast

    Packet *out = malloc(sizeof(Packet) + 1024); // 1024 byte payload max
    if (!out) return 0;

    parse_packet(data, size, out, 1024);

    free(out);
    return 0;
}

```

Chạy:

```

clang -fsanitize=address,fuzzer parse.c fuzz.c -o fuzz
./fuzz -max_len=200

```

ASan + libFuzzer: ASan detect memory bug, fuzzer drive input.

Verify với CBMC (chứng minh không bug với fix):

```

#include <assert.h>
#include <stdlib.h>

int main(void) {

```



```

size_t size = nondet_size_t();
__CPROVER_assume(size <= 10000); // bound for tractability

uint8_t *data = malloc(size);
if (!data) return 0;
// Fill data với nondet
for (size_t i = 0; i < size; i++) data[i] = nondet_uint8();

Packet *out = malloc(sizeof(Packet) + 1024);
if (out) {
    parse_packet(data, size, out, 1024);
    // Property: no memory corruption (automatic via --pointer-check)
}

free(data);
free(out);
return 0;
}

```

```
cbmc parse_harness.c --pointer-check --bounds-check --memory-leak-check
```

Bài học: parser binary format là **#1 attack surface**. Mỗi field length cần overflow-safe check. Format pattern: size_t consumed, tăng dần, compare với size_remaining.

Tóm tắt Cụm 4

| Sample | Topic | Tool |
|---------------------|---------------------------|-------------------------|
| 1 Coverage demo | Statement/branch coverage | gcov, llvm-cov |
| 2 MC/DC | Boolean coverage | gcov MC/DC mode |
| 3 Fuzz target naïve | Property-based | libFuzzer + ASan |
| 4 Symbolic exec | DSE for magic constant | CBMC, KLEE |
| 5 BMC test gen | Auto test generation | CBMC --cover |
| 6 Parser fuzz | Integer overflow check | libFuzzer + ASan + CBMC |

6 sample = blueprint cho testing modern: coverage + property + fuzz + formal.

Pattern fuzz harness tốt

Sau khi đọc, blueprint cho **fuzz harness chất lượng**:

1. **Limit input size:** if (size > MAX) return 0; để fuzzer fast.
2. **Multiple input formats:** parse data từ input theo schema, không treat as opaque blob.
3. **Property check:** sau gọi target, assert invariant (size bound, round-trip).
4. **Memory cleanup:** alloc trong harness phải free để fuzzer detect leak.
5. **Differential:** nếu có ref impl, compare output.
6. **Seed corpus:** cung cấp 10-100 valid input để fuzzer warm start.
7. **Dictionary:** với grammar-based format (HTTP, SQL), cung cấp keyword dictionary.

Harness tốt = bug found tăng 10x với cùng CPU time.

Combine BMC + Fuzz

Workflow modern:

1. **BMC** trên hot function □ tìm bug functional với formal guarantee (bounded).
2. **Fuzz** trên parser/codec □ tìm bug runtime với volume input.
3. **BMC sinh seed corpus** cho fuzz (sample 5).
4. **Fuzz crash** □ **minimize, reproduce, file** □ fix.
5. **Regression**: thêm crash input vào test suite, run mỗi CI.

Combination = highest signal-to-noise. Pure formal có scope limit; pure fuzz không guarantee.

Tiếp theo: [8.6 Exercise code analysis](#)

8.6 Phân tích Exercise Set 2 với tool

Tóm tắt một dòng: 7 bài trong **Exercise Set 2** đã có đáp án dạng manual review. Bài này quay lại từng bài với góc nhìn **tool automation**: chạy CBMC/ASan/UBSan để chứng minh bug, classify CWE, đánh giá CVSS, và liệt kê fix pattern. Mục tiêu: thấy mỗi bài thủ công có thể được automated trong CI.

Bài 1: sprintf buffer overflow

Code gốc (tóm tắt):

```
void greet(char *name) {
    char buf[16];
    sprintf(buf, "Hello, %s!", name);
    printf("%s\n", buf);
}
```

Phân tích

| Thuộc tính | Giá trị |
|----------------------|---|
| Bug class | Stack buffer overflow |
| CWE | CWE-121 Stack-based Buffer Overflow |
| CVSS 3.1 | 8.8 High (vector:
AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H) |
| Tool bắt được | Compiler warning, ASan, CBMC –bounds-check,
GCC FORTIFY_SOURCE |

Verify bằng tool

Method 1: ASan runtime

```
clang -fsanitize=address -g bai1.c -o bai1
./bai1 "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
```

Output:

```
=====
==12345==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x...
    #0 0x... in sprintf
    #1 0x... in greet bai1.c:5
SUMMARY: AddressSanitizer: stack-buffer-overflow bai1.c:5 in greet
```

ASan bắt ngay khi runtime.

Method 2: CBMC formal

Cần wrap thành harness vì argv[1] là từ environment:

```
#include <string.h>
#include <stdio.h>

void greet(char *name);
```

```
int main(void) {
    char name[20];
    for (int i = 0; i < 19; i++) name[i] = nondet_char();
    name[19] = '\0';
    greet(name);
    return 0;
}
```

```
cbmc harness.c bai1.c --bounds-check --pointer-check
```

[greet.array_bounds.1] line 5 array `buf' upper bound: FAILURE

Method 3: FORTIFY_SOURCE compile-time

```
gcc -O2 -D_FORTIFY_SOURCE=2 bai1.c -o bai1
./bai1 "AAAAAAAAAAAAAAAAAAAA"
```

Output:

```
*** buffer overflow detected ***: terminated
Aborted (core dumped)
```

FORTIFY_SOURCE replace sprintf bằng __sprintf_chk check size.

Bài học

1 dòng fix: sprintf → snprintf(buf, sizeof(buf), ...). ASan + FORTIFY_SOURCE bắt trong CI thường lệ.

Bài 2: Integer overflow trong calloc

Code gốc:

```
void* alloc_buffer(size_t count, size_t size) {
    size_t total = count * size;
    void *p = malloc(total);
    if (p == NULL) return NULL;
    memset(p, 0, total);
    return p;
}
```

Phân tích

| Thuộc tính | Giá trị |
|------------------|---|
| Bug class | Integer overflow trong allocation |
| CWE | CWE-190 Integer Overflow + CWE-680 |
| CVSS | 9.8 Critical (RCE potential nếu attacker control count, size) |
| Tool | UBSan, CBMC --unsigned-overflow-check |

Verify

UBSan:

```
clang -fsanitize=undefined,address bai2.c -o bai2
./bai2
```

UBSan bắt unsigned overflow runtime:

bai2.c:5:21: runtime error: unsigned integer overflow: 4294967296 * 8 cannot be represented

Wait, trên 64-bit thì $4294967296 * 8 = 34359738368$ không overflow size_t (64-bit max $\sim 1.8e19$).

UBSan **không bắt** vì không thực sự overflow.

Pattern này nguy hiểm trên **32-bit** (size_t = 4 byte) hoặc với operand lớn hơn:

```
alloc_buffer(0x100000000ULL, 0x10); // 0x10_00000000 trên 64-bit OK, nhưng
alloc_buffer(0xFFFFFFFFFFFFFFFFULL, 2); // overflow guaranteed
```

CBMC formal:

```
int main(void) {
    size_t count = nondet_size_t();
    size_t size = nondet_size_t();
    alloc_buffer(count, size);
    return 0;
}
```

```
cbmc harness.c bai2.c --unsigned-overflow-check
```

[alloc_buffer.overflow.1] line 5 arithmetic overflow on unsigned * in count * size: FAILURE

CBMC khám phá mọi count * size overflow.

Fix chuẩn: dùng __builtin_mul_overflow:

```
void* alloc_buffer(size_t count, size_t size) {
    size_t total;
    if (__builtin_mul_overflow(count, size, &total)) return NULL;
    void *p = malloc(total);
    if (p == NULL) return NULL;
    memset(p, 0, total);
    return p;
}
```

Hoặc đơn giản dùng calloc(count, size) (POSIX guarantee overflow check):

```
return calloc(count, size);
```

Bài học

calloc(n, s) chuẩn POSIX bắt buộc check overflow. Dùng calloc thay vì malloc(n * s). Custom allocator dùng __builtin_mul_overflow.

Bài 3: Uninitialized variable

Code gốc (giả định, pattern phổ biến):

```
int compute_total(int *items, int count) {
    int sum;
    for (int i = 0; i < count; i++) {
        sum += items[i];    // BUG: sum uninitialized
    }
    return sum;
}
```

Phân tích

| Thuộc tính | Giá trị |
|------------------|--|
| Bug class | Uninitialized read |
| CWE | CWE-457 Use of Uninitialized Variable |
| CVSS | 4-7 Medium-High (depends on what data leaks) |
| Tool | MSan, CBMC, compiler -Wuninitialized |

Verify

Compiler warning:

```
gcc -Wall -Wuninitialized bai3.c
```

bai3.c:5:9: warning: 'sum' is used uninitialized

Modern compiler bắt 95% case này.

MemorySanitizer:

```
clang -fsanitize=memory bai3.c -o bai3
./bai3
```

```
==12345==WARNING: MemorySanitizer: use-of-uninitialized-value
#0 in compute_total bai3.c:5
```

MSan bắt mọi uninitialized read runtime, kể cả khi compiler không thấy (e.g., heap memory chưa init).

CBMC:

CBMC track “uninit” qua propagation. Symbolic value của uninit variable không constrain, bất kỳ assertion về sum đều fail.

Fix

```
int sum = 0;    // explicit init
```

Hoặc designated init:

```
struct Config cfg = {0};    // t[] t c[] field = 0
```

C++ initialize tự động cho object có constructor. C không, phải nhớ init thủ công.

Bài học

Enable `-Wall -Wuninitialized` trong Cl. MSan run trên test suite catch heap case. Mọi declaration nên đi kèm initialization (constraint coding style).

Bài 4: Off-by-one trong loop

Code gốc (pattern):

```
void fill_array(int *arr, int n) {  
    for (int i = 0; i <= n; i++) {    // BUG: <= n, should be < n  
        arr[i] = 0;  
    }  
}
```

Phân tích

| Thuộc tính | Giá trị |
|------------------|---|
| Bug class | Off-by-one OOB |
| CWE | CWE-193 Off-by-one Error |
| CVSS | 7.5 High (memory corruption) |
| Tool | CBMC <code>--bounds-check</code> , ASan |

Verify

CBMC:

```
int main(void) {  
    int arr[10];  
    fill_array(arr, 10);  
    return 0;  
}
```

```
cbmc harness.c bai4.c --bounds-check
```

[fill_array.array_bounds.1] line 3 array `arr' upper bound: FAILURE

i = 10 ghi arr[10], out of bounds.

ASan: tương tự, bắt runtime.

Fix

```
for (int i = 0; i < n; i++)    // < thay vì <=
```

Pattern Loop “for i = 0 to n”: **luôn dùng <, không <=**.

Bài học

Off-by-one là 1 trong 3 bug “easy to write, hard to spot”: tested 100 lần có thể hit, hoặc không hit. CBMC bắt exhaustively trong bound.

Bài 5: Race condition đơn giản

Code gốc (pattern):

```
#include <pthread.h>

int counter = 0;

void* worker(void *arg) {
    for (int i = 0; i < 1000; i++) counter++;
    return NULL;
}

int main(void) {
    pthread_t t[4];
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, worker, NULL);
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
    assert(counter == 4000);    // FAIL
    return 0;
}
```

Phân tích

| Thuộc tính | Giá trị |
|------------------|--|
| Bug class | Data race |
| CWE | CWE-362 Concurrent Execution using Shared Resource |
| CVSS | Varies 4-9 (depends on consequence) |
| Tool | TSan, CBMC –pthread |

Verify

TSan:

```
clang -fsanitize=thread -g bai5.c -o bai5
./bai5
```

```
WARNING: ThreadSanitizer: data race
  Write of size 4 at 0x... by thread T2:
    #0 worker bai5.c:7
  Previous write of size 4 at 0x... by thread T1:
    #0 worker bai5.c:7
```

TSan detect race ngay lần chạy đầu.

CBMC:

```
cbmc bai5.c --pthread --unwind 3
```

CBMC khám phá interleaving, bắt counter < 4000 schedule.

Fix

```
#include <stdatomic.h>
atomic_int counter = 0;

void* worker(void *arg) {
    for (int i = 0; i < 1000; i++) atomic_fetch_add(&counter, 1);
    return NULL;
}
```

Bài học

TSan = first line of defense cho race. Run unit test với TSan trong CI. CBMC formal cho critical concurrent code.

Bài 6: Use-after-free trong linked list

Code gốc (pattern):

```
typedef struct Node { int val; struct Node *next; } Node;

void process_list(Node *head) {
    Node *curr = head;
    while (curr != NULL) {
        free(curr);
        curr = curr->next;    // BUG: UAF read
    }
}
```

Phân tích

| Thuộc tính | Giá trị |
|------------------|--|
| Bug class | Use After Free |
| CWE | CWE-416 Use After Free |
| CVSS | 9.8 Critical (memory corruption → RCE) |
| Tool | ASan, CBMC –pointer-check |

Verify

ASan:

```
clang -fsanitize=address bai6.c -o bai6
./bai6
```

```
==12345==ERROR: AddressSanitizer: heap-use-after-free
    READ of size 8 at 0x... thread T0
    #0 process_list bai6.c:7
```

ASan track free, bắt subsequent read.

CBMC:

CBMC track allocation, bắt UAF qua object_id và validity bit.

Fix

```
void process_list(Node *head) {
    Node *curr = head;
    while (curr != NULL) {
        Node *next = curr->next;    // save next BEFORE free
        free(curr);
        curr = next;
    }
}
```

Bài học

Pattern “traverse and free”: **lưu next trước khi free current**. Đây là idiom chuẩn cho linked list cleanup.

Bài 7: Struct padding và alignment

Code gốc (từ exercise 7):

```
struct Foo {
    char a;
    int b;
    char c;
};

int main(void) {
    printf("sizeof(struct Foo) = %zu\n", sizeof(struct Foo));
    return 0;
}
```

Phân tích

Đây không phải bug, mà là **đặc tính ABI** của C: compiler chèn padding để align field.

Layout trên x86-64:

Offset 0: a (char, 1 byte)
Offset 1–3: PADDING (3 byte, để int align 4)
Offset 4–7: b (int, 4 byte)
Offset 8: c (char, 1 byte)
Offset 9–11: PADDING (3 byte, struct size align 4)
Total: 12 byte

sizeof(struct Foo) = 12, không phải 1+4+1 = 6.

Pitfall

Padding gây vấn đề khi:

1. **Network serialize**: send raw struct qua socket □ padding gửi cả □ leak memory data.
2. **Memcmp**: memcmp(&foo1, &foo2, sizeof(foo)) compare padding cũng □ false negative.
3. **Struct hash**: hash raw bytes □ padding ảnh hưởng □ cùng data, khác hash.

Fix patterns

Pattern 1: pack struct (loại bỏ padding):

```
struct __attribute__((packed)) Foo {
    char a;
    int b;
    char c;
};
sizeof = 6
```

Nhưng access unaligned int □ slow trên x86, crash trên ARM strict.

Pattern 2: reorder field theo size decreasing (tự nhiên align):

```
struct Foo {
    int b;        // 4 byte
    char a;       // 1 byte
    char c;       // 1 byte
    char pad[2];  // explicit pad
};
sizeof = 8
```

Pattern 3: explicit serialize:

```
void serialize_foo(struct Foo *f, uint8_t *buf) {
    buf[0] = f->a;
    memcpy(buf + 1, &f->b, 4);
    buf[5] = f->c;
}
```

Đảm bảo wire format independent of compiler padding.

Bài học

Struct layout có **padding implicit**. Network code, hash, memcmp phải aware. Tool pahole (Linux) print struct layout chi tiết.

Bảng tóm tắt 7 bài

| Bài | Bug class | CWE | CVSS | Tool đề xuất |
|-----|------------------|---------|---------|-------------------------------------|
| 1 | Buffer overflow | CWE-121 | 8.8 H | ASan, snprintf, FORTIFY |
| 2 | Integer overflow | CWE-190 | 9.8 C | UBSan, CBMC, __builtin_mul_overflow |
| 3 | Uninitialized | CWE-457 | 4-7 M-H | MSan, -Wuninitialized |
| 4 | Off-by-one | CWE-193 | 7.5 H | CBMC, ASan |

| Bài | Bug class | CWE | CVSS | Tool đề xuất |
|-----|-------------|----------|--------|------------------------|
| 5 | Data race | CWE-362 | varies | TSan, CBMC
-pthread |
| 6 | UAF | CWE-416 | 9.8 C | ASan, smart
pointer |
| 7 | Padding ABI | (no CWE) | - | pahole, code
review |

7 bài này = 7 CWE category quan trọng nhất cho C/C++.

CI workflow tích hợp

Đề xuất `.github/workflows/security.yml` cho project C/C++:

```
name: Security checks
on: [push, pull_request]

jobs:
  asan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: clang -fsanitize=address,undefined -g src/*.c -o test
      - run: ./test

  msan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: clang -fsanitize=memory -g src/*.c -o test
      - run: ./test

  tsan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: clang -fsanitize=thread -g src/*.c -o test
      - run: ./test

  cbmc:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: sudo apt install cbmc
      - run: |
          for f in src/*_harness.c; do
            cbmc "$f" --bounds-check --pointer-check --signed-overflow-check
          done
```

4 job song song, chạy ~5 phút. Block PR nếu fail. Catch ~95% bug class trong 7 bài tập trên.

Tóm tắt cụm 8 (Code Analysis)

Sau khi đọc **5 bài** trong cụm này, bạn đã review:

- **10 sample Cụm 1** (vulnerabilities catalog).
- **8 sample Cụm 2** (BMC encoding).
- **8 sample Cụm 3** (concurrency).
- **6 sample Cụm 4** (testing/fuzzing).
- **7 bài exercise** (CWE categorization).

Tổng cộng **39 code pattern** với analysis chi tiết, fix gợi ý, tool verification. Bạn giờ có:

1. **Vocabulary CWE/CVSS** cho công việc thực tế (CVE triage, security audit).
2. **Reflexes review code** với checklist 5-6 điểm.
3. **Knowledge tool stack**: CBMC, ASan, MSan, TSan, UBSan, libFuzzer, FORTIFY_SOURCE.
4. **CI integration blueprint** áp dụng được ngay.
5. **Pattern fix** cho mỗi class bug (RAII, smart pointer, snprintf, builtin_overflow).

Đây là **kỹ năng practical** quan trọng nhất sau khi đã có vocabulary từ Lec 1-7.

Kết thúc Cụm 8 (Code Analysis). Bạn đã hoàn thành toàn bộ curriculum: 7 cụm lý thuyết + 1 cụm phân tích code thực hành. Quay về **trang chính** hoặc xem **Cross-reference** để điều hướng nâng cao.